LatentView Analytics

COMPLETE TECHNICAL QUESTION & CODING BANK

> **Updated Version:** This document includes the fundamental conceptual questions along with the core coding/programming problems (SQL, Python, Spark) typically tested during LatentView's technical assessment rounds.

1. Data Analyst / Senior Data Analyst Coding

Focus: SQL Window functions, aggregations, and data preparation

- **SQL Challenge - Second Highest Salary:** Write an ANSI SQL query to find the second-highest salary from an `Employee` table without relying on vendor-specific keywords like `LIMIT` or `TOP`.

```
-- Solution Framework
SELECT MAX(Salary)
FROM Employee
WHERE Salary < (SELECT MAX(Salary) FROM Employee);
```

- **SQL Challenge - Window Functions:** Given a sales table `orders (order_id, customer_id, order_date, amount)`, write a query to find the top 2 highest spending orders for every single customer.

```
WITH RankedSales AS (
    SELECT customer_id, order_id, amount,
           DENSE_RANK() OVER(PARTITION BY customer_id ORDER BY amount DESC) as rnk
    FROM orders
)
SELECT customer_id, order_id, amount FROM RankedSales WHERE rnk <= 2;
```

2. Data Scientist Coding

Focus: Python data structure manipulation and basic algorithmic thinking

- **Python Challenge - Two Sum Variation:** Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. Optimize for time complexity.

```
def two_sum(nums, target):
    lookup = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in lookup:
            return [lookup[complement], i]
        lookup[num] = i
    return []
```

- **Python Challenge - String Parsing:** Write a Python function to check whether a given string is a valid palindrome, ignoring spaces and capitalization.

3. Senior Data Scientist Coding

Focus: Custom algorithm implementations and data engineering logic inside ML models

- **Python ML Implementation:** Write a pure Python function to compute the **Log-Loss** evaluation metric given an array of true binary targets y and predicted probabilities $\hat{y}$ without importing NumPy or Scikit-learn.

```
import math
def compute_log_loss(y_true, y_pred):
    epsilon = 1e-15
    loss = 0.0
    for y, p in zip(y_true, y_pred):
        p = max(epsilon, min(1 - epsilon, p)) # Clip values
        loss += y * math.log(p) + (1 - y) * math.log(1 - p)
    return -loss / len(y_true)
```

- **Feature Optimization:** Write a function to identify and filter out highly collinear pairs of features based on an input covariance or correlation matrix dataframe.

4. Data Engineer Coding

Focus: PySpark DataFrame manipulation, memory limits, and custom ingestion handling

- **PySpark - Cumulative Session Calculation:** Given a PySpark dataframe tracking user activity clicks `(user_id, timestamp, page)`, write code to calculate the running total count of clicks per user sorted chronologically.

```
from pyspark.sql import Window
import pyspark.sql.functions as F

windowSpec =
Window.partitionBy("user_id").orderBy("timestamp").rowsBetween(Window.unboundedPreceding,
Window.currentRow)
df_with_running_total = df.withColumn("running_clicks",
F.count("page").over(windowSpec))
```

- **Python - Resilient Ingestion API Parser:** Write a Python script using `requests` that pulls paginated data from an API endpoint, catches connection timeout errors gracefully, and drops payload batches dynamically into a localized staging parquet file structure.

5. Data Science Specialist

Focus: Systems coding architecture and algorithmic trade-offs

- **System Execution Evaluation:** Suppose you are profiling a memory-intensive clustering loop. Implement a custom Python decorator function to track and log the peak memory allocation and execution runtime duration of any arbitrary algorithm block.

End of Coding Supplement • LatentView Analytics Interview Guide.